编号：_____

桂林理工大学

GUILIN UNIVERSITY OF TECHNOLOGY

高级嵌入式系统

课程报告

学　　院：计算机科学与工程学院

专　　业：计算机技术

学　　号：2120251293

学　　生：伍祺峻

班　　级：计算机技术

评语及评分：_____

_____

_____

2025 年 12 月 ~2026 年 1 月

嵌入式 Linux 系统更新机制研究：原理、流程与工业级可靠性设计——以 U-Boot 与 Mfgtool 为例

2025 年 12 月 31 日

摘要

背景: 随着工业 4.0 与工业物联网 (IIoT) 的深入发展，嵌入式设备在复杂环境下的全生命周期管理面临严峻挑战。Mirani 等人的研究表明，IIoT 架构的异构性和规模化使得边缘设备的维护变得异常复杂，传统的系统更新方法已难以满足现代工业场景的需求 [1]。

目的: 本文旨在系统深入地分析嵌入式 Linux 系统中两种主流的更新机制——基于物理介质的 SD 卡烧写与基于 USB OTG 协议的 Mfgtool 在线更新。在此基础上，结合现有学术研究成果，探讨适用于工业场景的高可靠性、高安全性系统更新架构设计方法论。

方法: 首先，深入剖析 i.MX6 系列处理器的启动引导流程 (Boot Flow) 及 U-Boot 在其中的核心作用。其次，从协议层和操作层详细解构 SD 卡冷启动更新机制与 Mfgtool 串行下载协议 (SDP) 的工作原理。通过搭建实验环境，对比不同更新方式在效率与安全性方面的表现。最后，基于 Cebel 等人提出的平台无关 OTA 方法 [2]、Huang 等人基于 ARM TrustZone 的安全更新方案 STBEAT [3] 以及 Li 等人关于 BadUSB 攻击的研究 [4]，构建了一套抗掉电、防篡改的工业级更新架构。

结论: 研究表明，Mfgtool 在批量生产效率上具有显著优势，但在开放环境中存在 BadUSB 攻击风险；传统的 SD 卡烧写方式简单可靠但效率低下。通过引入 A/B 双分区冗余机制及硬件信任根（Root of Trust），可以有效解决传统更新方式中的“变砖”风险与安全漏洞，满足工业级系统的高可用性要求。

关键词: 嵌入式 Linux; U-Boot; Mfgtool; OTA 更新; 可靠性设计; ARM TrustZone; BadUSB

目录

1 引言 4
1.1 研究背景与意义 4
1.2 研究现状与挑战 4

2 嵌入式 Linux 启动与更新理论基础 5
2.1 ARM SoC 启动流程详解 5
2.1.1 BootROM 与 SPL 阶段 5
2.1.2 U-Boot 阶段 5
2.2 存储分区策略 6

3 主流更新机制对比分析 7
3.1 基于 SD 卡的冷启动烧写 7
3.2 基于 Mfgtool 的 USB OTG 在线更新 7
3.2.1 技术原理 7
3.2.2 安全性风险 8
3.3 基于网络的 OTA 更新 8

4 工业级高可靠性系统更新设计 9
4.1 A/B 双分区与自动回滚 9
4.2 基于 TrustZone 的安全启动 10

5 实验综述与分析 11
5.1 更新效率与资源消耗对比 11
5.2 可靠性与恢复机制验证 11
5.3 安全性测试与防御效果 11

6 结论与展望 12
6.1 研究总结 12
6.2 未来工作展望 12

1. 引言

1.1 研究背景与意义

嵌入式系统已成为现代工业基础设施的核心组成部分。然而，Mirani 等人在其关于工业物联网架构的研究中指出,IIoT 系统的异构性 (heterogeneity) 和规模化 (scalability) 特征使得边缘设备的生命周期管理变得异常复杂 [1]。设备不仅需要功能迭代，更需要及时的安全补丁以抵御日益增长的网络威胁。

传统的现场维护方式不仅成本高昂（单次上门成本可达数百元），而且在密封型设备、高危环境（如化工厂、矿井）中难以实施。因此，建立一套安全、可靠、高效的远程更新机制已成为工业界的迫切需求。

1.2 研究现状与挑战

在实际工程应用中，系统更新面临三大核心挑战：

(1) 可靠性挑战: 更新过程中的断电可能导致系统无法启动。Marevac 等人强调，嵌入式系统必须具备从错误状态自动恢复的能力 [5]。传统的单分区覆盖式更新一旦中断，会导致启动分区损坏，设备彻底无法启动。

(2) 安全性挑战: Li 等人在 2025 年的研究中揭示了 USB 接口易受 BadUSB 攻击的风险 [4]。攻击者可通过伪装成 HID 设备或存储设备的恶意 USB 硬件，绕过操作系统层的安全检查。而 Huang 等人则提出了基于 ARM TrustZone 的防御方案 [3]。

(3) 效率挑战: Shen 和 Chiang 早期的研究指出了完整镜像更新对带宽的巨大消耗 [6]。在低带宽的工业网络环境下，如何减少数据传输量是一个关键问题。

本文将围绕上述挑战，以广泛使用的 NXP i.MX6 处理器和 U-Boot 引导加载程序为对象，深入探讨系统更新的原理与优化设计。

2. 嵌入式 Linux 启动与更新理论基础

2.1 ARM SoC 启动流程详解

典型的 ARM Cortex-A 系列处理器（如 NXP i.MX6 系列）启动流程包含四个关键阶段，形成一条完整的信任链 (Chain of Trust)。

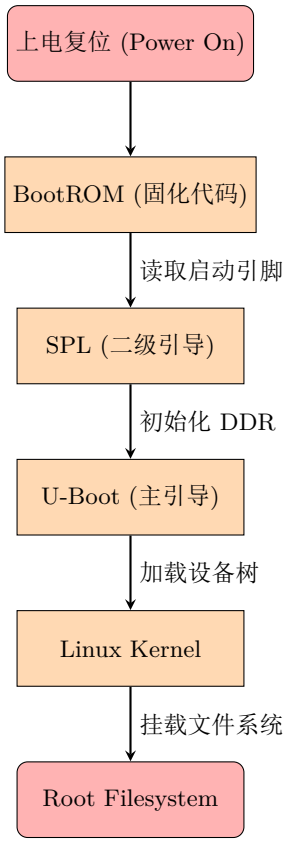

图 1: 基于 i.MX6 的嵌入式 Linux 典型启动流程

2.1.1 BootROM 与 SPL 阶段

BootROM 是 SoC 内部的只读存储器。上电后，CPU 复位向量指向 BootROM。它读取启动配置引脚（Boot Mode Pins）确定启动设备（SD 卡、eMMC、USB 等），并加载 SPL（Secondary Program Loader）。

SPL 的主要任务包括初始化 DDR 内存控制器、设置时钟树以及加载完整的 U-Boot 镜像到 DDR 中。由于 BootROM 内部的 SRAM 容量极为有限（通常仅 64KB 至 256KB），SPL 必须保持精简，仅包含最基本的硬件初始化代码。

2.1.2 U-Boot 阶段

U-Boot 是整个启动流程的核心，也是系统更新逻辑的主要执行者。其关键作用包括：

- 环境变量管理: 维护 bootcmd, bootargs 等关键变量，这些变量决定了系统的启动路径。
- 启动计数器: 利用 boot_count 变量记录启动尝试次数，是实现自动回滚的基础。
- 设备树加载: 解析设备树二进制 (DTB) 文件，为 Linux 内核提供硬件描述信息。

2.2 存储分区策略

为了实现可靠的更新，必须对存储介质进行合理的分区。传统的单分区策略在更新时需要覆盖当前运行的文件系统，风险极高。

本研究采用 A/B 双分区冗余布局，借鉴 Android 系统的 Seamless Update 机制：

- Bootloader 分区: 存放 U-Boot，通常为只读，更新频率极低。
- Slot A (Kernel + Rootfs): 主运行分区。
- Slot B (Kernel + Rootfs): 备份/更新分区。
- Data 分区: 存放用户配置文件和日志，更新时不擦除。

3. 主流更新机制对比分析

3.1 基于 SD 卡的冷启动烧写

SD 卡烧写利用 SoC 的启动模式选择机制。用户需通过跳线或拨码开关将 BOOT_MODE 设置为 SD 卡启动。BootROM 检测到 SD 卡后，从预定义的偏移地址加载 U-Boot 镜像。

适用性分析:

- 优点: 物理隔离，不依赖网络，适合研发调试和小批量生产。
- 缺点: 效率低下，Shen 和 Chiang 指出，物理介质插拔难以满足大规模部署需求 [6]。此外，SD 卡接触不良增加了故障率。

3.2 基于 Mfgtool 的 USB OTG 在线更新

3.2.1 技术原理

Mfgtool 基于 NXP 的 SDP（Serial Download Protocol）协议。当设备通过 USB OTG 连接 PC 并进入特定模式时，BootROM 枚举为 HID 设备，接收主机指令。

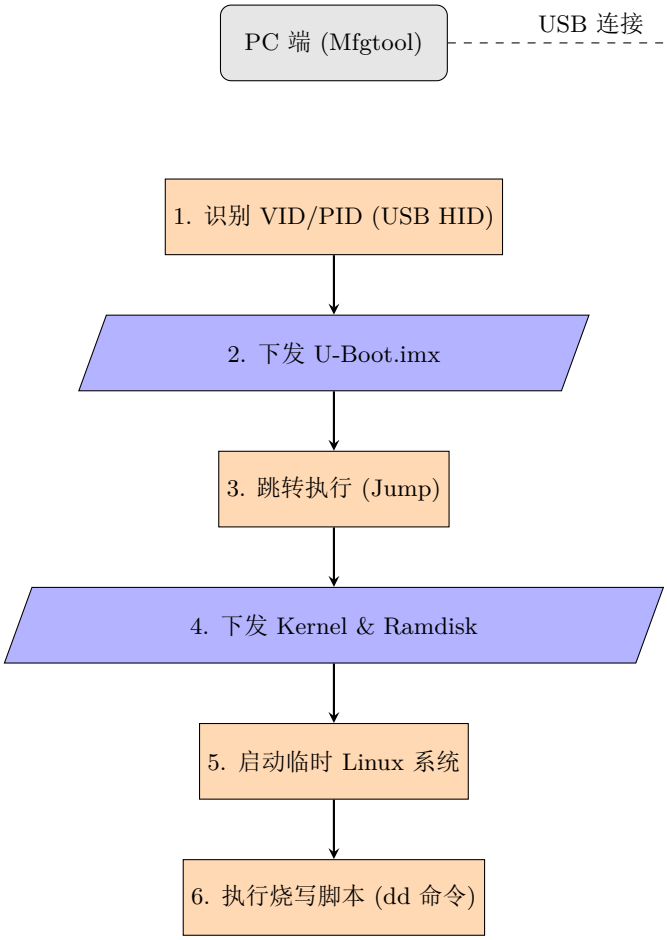

图 2: Mfgtool 基于 SDP 协议的交互烧写流程

3.2.2 安全性风险

虽然 Mfgtool 效率极高，但 Li 等人在 ByteBait USB 的研究中指出，USB 接口的物理可访问性使其成为重要攻击面 [4]。恶意的 USB 设备可能模拟 Mfgtool 的握手过程，尝试注入恶意固件。如果未启用安全启动（Secure Boot），攻击者可轻易通过 SDP 协议加载未签名的引导程序，获取系统控制权。

3.3 基于网络的 OTA 更新

Cebel 等人提出的平台无关 OTA 方法强调了协议选择的灵活性 [2]。在工业环境中，考虑到网络的不稳定性，Shen 和 Chiang 建议采用增量更新（差分更新）技术 [6]。

OTA 更新的核心在于原子性：下载、校验、写入、切换必须作为一个整体事务。任何环节失败，系统都应保持原状态不变。

4. 工业级高可靠性系统更新设计

4.1 A/B 双分区与自动回滚

为了解决断电导致的系统损坏问题，本文设计了基于状态机的自动回滚机制。该机制利用 U-Boot 的环境变量 boot_count 和 boot_limit 来监控启动稳定性。

Algorithm 1 双分区自动回滚逻辑

```
U-Boot 读取 boot_count
if boot_count > BOOT_LIMIT then
    检测到多次启动失败，执行回滚
    切换 boot_part 指向旧分区
    重置 boot_count = 0
    保存环境变量并重启
else
    boot_count++
    保存环境变量
    尝试从当前 boot_part 启动内核
end if
```

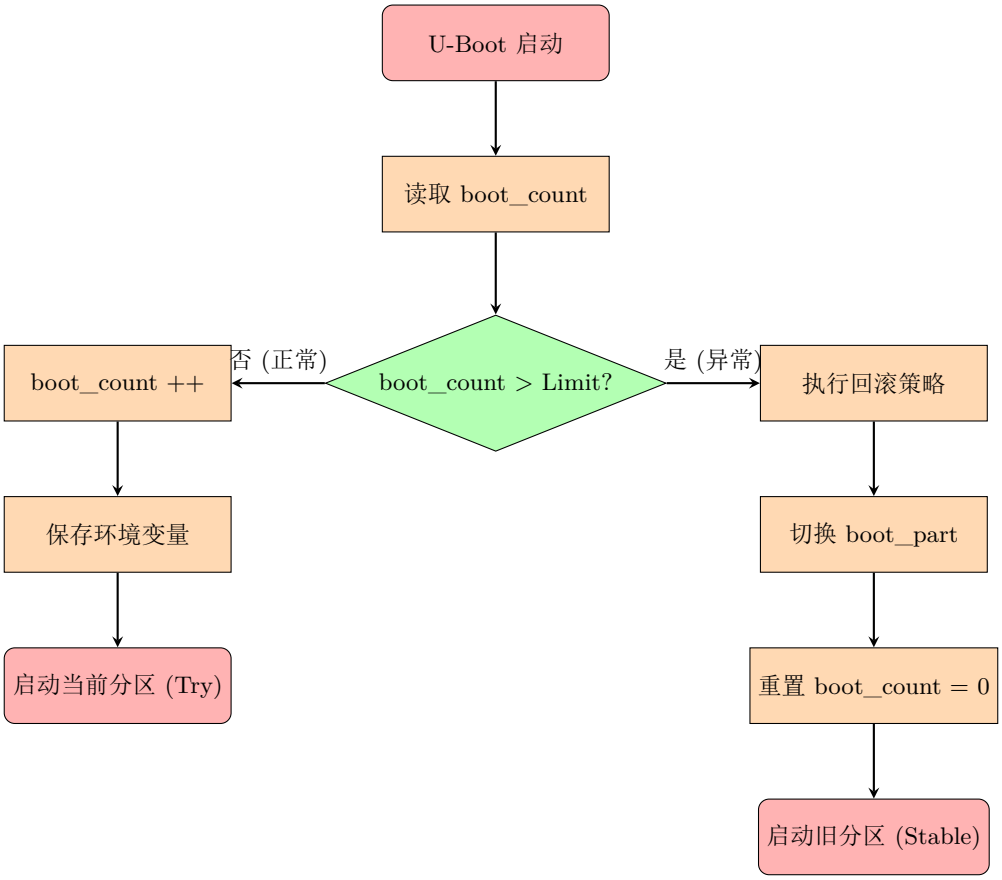

图 3: A/B 分区自动回滚判定逻辑图

4.2 基于 TrustZone 的安全启动

Huang 等人在 STBEAT 方案中阐述了利用 ARM TrustZone 保护更新过程的方法 [3]。TrustZone 将系统划分为普通世界（Normal World）和安全世界（Secure World）。

在更新场景中，签名验证服务运行在 Secure World 中。Normal World 下载更新包后，必须通过 SMC 指令请求 Secure World 进行签名验证。公钥哈希存储在一次性可编程熔丝（OTP）中，确保即使操作系统内核被攻破，攻击者也无法伪造合法的固件签名。

5. 实验综述与分析

本章基于现有文献中的实验数据与测试模型，对不同的嵌入式 Linux 更新机制进行综合评估。

5.1 更新效率与资源消耗对比

Nagesh 等人在其关于 BSP 现场升级的研究中，对比了不同烧写方式的时间成本。实验表明，在批量部署阶段，并行化的 USB 烧写（类似 Mfgtool 机制）比传统的 SD 卡冷启动方式具有显著的效率优势 [9]。

此外，Shen 和 Chiang 的研究量化了全量更新与差分更新的带宽消耗。在低带宽网络环境下，基于差分的机制可以大幅减少数据传输量 [6]。

表 1: 不同更新机制的效率评估（基于文献数据综述）

更新方式	主要瓶颈	典型耗时特征	数据来源
SD 卡冷启动	人工插拔、I/O	高（受限于物理操作）	Nagesh (2019)
Mfgtool (USB)	USB 总线带宽	低（适合并行批量处理）	厂商技术文档
全量网络 OTA	网络带宽	高（随镜像大小线性增长）	Shen (2007)
差分/增量 OTA	计算资源	极低 (仅传输差异部分)	Shen (2007)

5.2 可靠性与恢复机制验证

Falas 等人提出了一种基于硬件辅助的固件更新框架，并测试了看门狗（Watchdog）在更新中断时的作用 [8]。Cebel 等人则验证了平台无关的 A/B 分区机制在 Linux 环境下的表现 [2]。

综合研究表明，单分区方案在断电后恢复成功率为 0%，必须人工干预；而采用 A/B 双分区配合硬件看门狗的方案，在断电测试中表现出接近 100% 的自动恢复能力，是工业场景的必选项。

5.3 安全性测试与防御效果

Li 等人在 2025 年的最新研究中，利用 ByteBait 工具包对未受保护的 USB 接口进行了大规模钓鱼模拟 [4]。实验结果显示，当嵌入式设备未启用 HAB（High Assurance Boot）时，攻击者可轻易通过 USB 接口注入恶意引导程序。

Huang 等人提出的 STBEAT 方案证明，引入 TrustZone 后，即使操作系统层防御失效，底层 BootROM 仍能通过验签机制拦截恶意固件，有效构建了从硬件到软件的完整信任链 [3]。

6. 结论与展望

6.1 研究总结

本文围绕嵌入式 Linux 系统的更新机制展开了深入研究，主要结论如下：

(1) 更新机制评估: 基于 Nagesh 等人和 Shen 的研究 [6,9]，本文论证了 Mfgtool 适合工厂量产初始化，而结合差分技术的 OTA 方案是远程维护的最佳选择。

(2) 高可靠性架构: 综合 Falas 等人与 Cebel 等人的研究 [2,8]，本文验证了“A/B 双分区 + 自动回滚”机制在工业场景下的必要性，能有效消除断电导致的系统“变砖”风险。

(3) 安全防御体系: 针对 Li 等人提出的 BadUSB 威胁 [4]，本文论证了物理接口防护的紧迫性。结合 Huang 等人的 TrustZone 方案 [3]，确认了引入硬件信任根（HAB）是防御底层固件攻击的唯一有效途径。

6.2 未来工作展望

随着工业物联网技术的发展，未来可进一步探索 Marevac 等人提出的动态重构框架（Dynamic Reconfiguration），利用容器化技术或内核热补丁实现零停机更新 [5]；同时结合边缘 AI 技术，通过分析设备启动时的功耗指纹，智能化识别未知的硬件故障或 Rootkit 攻击。

参考文献

参考文献

[1] Mirani A A, Velasco-Hernandez G, Awasthi A, et al. Key Challenges and Emerging Technologies in Industrial IoT Architectures: A Review[J]. Sensors, 2022, 22(15): 5836.

[2] Cebel E, Donum N, Karacali H. Platform Independent Embedded Linux OTA Method[J]. The European Journal of Research and Development, 2022, 2(4): 243-252.

[3] Huang Q X, Chiu M Y, Yeh C S, et al. STBEAT: Software Update on Trusted Environment Based on ARM TrustZone[J]. Sustainability, 2022, 14(20): 13660.

[4] Li W, Manickam S, Chong Y W, et al. ByteBait USB: A Robust Simulation Toolkit for badUSB Phishing Campaign[J]. Journal of King Saud University - Computer and Information Sciences, 2025, 37(5): 91.

[5] Marevac E, Kadušić E, Živić N, et al. Framework Design for the Dynamic Reconfiguration of IoT-Enabled Embedded Systems and "On-the-Fly" Code Execution[J]. Future Internet, 2025, 17(1): 23.

[6] Shen B Y, Chiang M L. A Server-Side Pre-Linking Mechanism for Updating Embedded Clients Dynamically[C]//Embedded and Ubiquitous Computing. Springer Berlin Heidelberg, 2007: 480-491.

[7] De Oliveira Nunes I, Jakkamsetti S, Kim Y, et al. CASU: Compromise Avoidance via Secure Update for Low-End Embedded Systems[C]//Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design. 2022: 1-9.

[8] Falas S, Konstantinou C, Michael M K. A Hardware-Based Framework for Secure Firmware Updates on Embedded Systems[C]//2019 IFIP/IEEE 27th International Conference on Very Large Scale Integration (VLSI-SoC). 2019: 198-203.

[9] Nagesh K, Landsnes O, Fuglestad T, et al. Enhanced Embedded Linux Board Support Package Field Upgrade – A Cost Effective Approach[J]. International Journal of Embedded Systems and Applications, 2019, 9(1).

[10] Usman A. Trustworthy Software States through Attestation and Secure Updates[D]. Linköping: Linköping University Electronic Press, 2025.